

API

API (Application Programming Interface) is a way for one software system to communicate with another.

Think of an API as a messenger between two applications.

The style decides three things: how a request is shaped, how the data is encoded, and what the connection looks like.

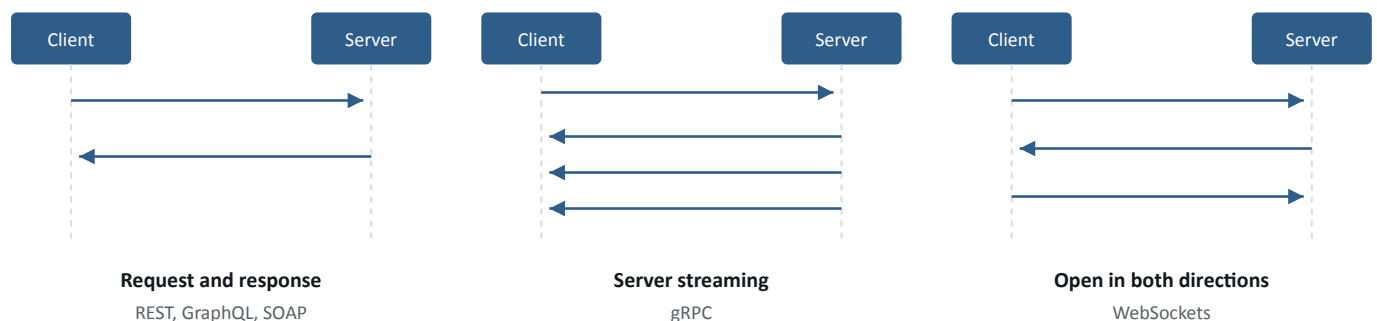
A good order to learn them in:

What an API is → REST → gRPC → GraphQL → WebSockets → SOAP
→ Choosing → Caching → Versioning → Circuit breaker

1. What an API is

Two programs need to agree before they can talk. The agreement covers the address to send to, the shape of the request, the shape of the reply, and what happens when something fails.

The styles below are different answers to that same problem. They differ most in the shape of the conversation.



The first shape closes the connection after each reply. The third keeps it open, so the server can speak first.

2. REST

REST (Representational State Transfer) is an architectural style for designing APIs that use HTTP to communicate with resources.

Every resource has an address. The HTTP method says what to do with it, and the body is usually JSON.

Method	What it does	Safe to repeat
GET	Reads a resource	Yes, and it changes nothing
POST	Creates a resource	No, it creates another one
PUT	Replaces a resource	Yes, the result is the same
PATCH	Changes part of a resource	Usually, depends on the body
DELETE	Removes a resource	Yes, it stays deleted

Reading one employee:

```
GET /employees/1 HTTP/1.1
Host: api.example.com
Accept: application/json
```

Response:

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "id": 1,
  "name": "Ravi",
  "department": "Engineering",
  "salary": 75000.00
}
```

Creating one:

```
POST /employees HTTP/1.1
Content-Type: application/json

{ "name": "Sara", "department": "Engineering", "salary": 71000.00 }
```

Response:

```
HTTP/1.1 201 Created
Location: /employees/7
```

The status code carries the outcome, so the caller does not have to read the body to know what happened:

Code	Meaning
200 OK	The request worked

Code	Meaning
201 Created	A new resource exists, <code>Location</code> says where
204 No Content	It worked and there is nothing to return
400 Bad Request	The request is malformed
401 Unauthorized	We do not know who is asking
403 Forbidden	We know, and they are not allowed
404 Not Found	No such resource
409 Conflict	It clashes with the current state
500 Internal Server Error	The server failed

Idempotency is asked about often. A request is idempotent when sending it twice leaves the system in the same state as sending it once. `GET`, `PUT` and `DELETE` are. `POST` is not, which is why a retried payment can charge twice unless the request carries a key the server remembers.

REST → resources with addresses, HTTP methods as verbs

REST is the default for a public API. It needs no special client, every language speaks HTTP, and responses cache well. The cost is that the server decides the shape of the response, so a client that needs three related things makes three calls.

3. gRPC

gRPC is a Remote Procedure Call (RPC) framework for calling a method on another service as though it were local.

Instead of resources, we call methods. The contract lives in a `.proto` file, and both sides generate code from it.

```

syntax = "proto3";

service EmployeeService {
    rpc GetEmployee (GetEmployeeRequest) returns (Employee);
    rpc ListEmployees (ListEmployeesRequest) returns (stream Employee);
}

message GetEmployeeRequest {
    int32 id = 1;
}

message Employee {
    int32 id = 1;
    string name = 2;
    string department = 3;
    double salary = 4;
}

```

The generated Java client is a normal method call:

```

EmployeeServiceBlockingStub stub = EmployeeServiceGrpc.newBlockingStub(channel);

Employee employee = stub.getEmployee(
    GetEmployeeRequest.newBuilder().setId(1).build());

System.out.println(employee.getName());

```

Output:

```
Ravi
```

The numbers after each field are field tags, not values. They are what the binary format uses on the wire, which is why renaming a field is safe and reusing a tag number is not.

gRPC supports four call shapes:

Shape	Request	Response
Unary	One	One
Server streaming	One	A stream
Client streaming	A stream	One
Bidirectional	A stream	A stream

gRPC → methods with a generated contract, binary over HTTP/2

It is fast, the payload is small, and the contract is checked at compile time. The cost is that a browser cannot call it directly without a proxy, and the binary body cannot be read in a log without tooling.

4. GraphQL

GraphQL is a query language and API technology that lets clients request exactly the data they need from a server.

There is one endpoint. The server publishes a schema, and the query has the shape of the answer.

```
type Employee {  
  id: ID!  
  name: String!  
  department: Department  
  salary: Float  
}  
  
type Department {  
  id: ID!  
  name: String!  
}  
  
type Query {  
  employee(id: ID!): Employee  
}
```

A query asking for two fields and one nested field:

```
query {  
  employee(id: 1) {  
    name  
    department {  
      name  
    }  
  }  
}
```

Response:

```
{
  "data": {
    "employee": {
      "name": "Ravi",
      "department": { "name": "Engineering" }
    }
  }
}
```

Nothing else comes back. No `id`, no `salary`, because the query did not ask for them.

That solves two problems REST has. Over-fetching is receiving fields we do not need. Under-fetching is having to make a second call for related data, which the nested `department` avoided here.

The costs are real. Every response is `200 OK` with errors inside the body, so HTTP status codes stop being useful. Caching is harder, because every query is a different shape. And a nested query can become one database call per row, the N+1 problem, unless the resolvers batch their loads.

GraphQL → one endpoint, the client picks the fields

5. WebSockets

WebSocket is a communication protocol that provides a persistent, two-way connection between a client and a server.

After it is set up, either side can send a message at any time without being asked.

It starts as an ordinary HTTP request that asks to change protocol:

```
GET /prices HTTP/1.1
Host: api.example.com
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Key: dGhlIHNhbXBsZSBub25jZQ==
Sec-WebSocket-Version: 13
```

The server agrees, and the connection stops being HTTP:

```
HTTP/1.1 101 Switching Protocols
Upgrade: websocket
Connection: Upgrade
```

A handler in Spring:

```

@Component
public class PriceHandler extends TextWebSocketHandler {

    @Override
    public void handleTextMessage(WebSocketSession session, TextMessage message)
        throws IOException {
        System.out.println("Received: " + message.getPayload());
        session.sendMessage(new TextMessage("ack"));
    }
}

```

WebSockets suit live prices, chat, notifications and collaborative editing: anything where the server has news the client did not ask for.

The cost is that the connection is stateful. A load balancer has to keep each client on the same server, and a reconnection has to rebuild whatever the session held.

When only the server sends, Server-Sent Events are simpler: one direction, plain HTTP, and the browser reconnects on its own.

WebSockets → one open connection, either side speaks

6. SOAP

SOAP (Simple Object Access Protocol) is a messaging protocol that wraps every message in an XML envelope.

The contract is a WSDL file, which describes every operation and type.

```

<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <getEmployee xmlns="http://example.com/employees">
      <id>1</id>
    </getEmployee>
  </soap:Body>
</soap:Envelope>

```

Response:

```

<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <getEmployeeResponse xmlns="http://example.com/employees">
      <employee>
        <id>1</id>
        <name>Ravi</name>
        <department>Engineering</department>
      </employee>
    </getEmployeeResponse>
  </soap:Body>
</soap:Envelope>

```

SOAP is older than the others and still runs in banking, insurance, telecom and government systems. It is worth knowing because those integrations are real work, not because a new service would choose it.

What it brings: a strict machine-readable contract, and a set of agreed standards on top for security, reliable delivery and transactions across services. What it costs: the envelope is heavy, XML parsing is slow next to JSON, and it is awkward to call by hand.

SOAP → XML envelope, WSDL contract, enterprise standards

7. Choosing between them

	REST	gRPC	GraphQL	WebSockets	SOAP
Data format	JSON	Binary	JSON	Any	XML
Transport	HTTP	HTTP/2	HTTP	TCP after upgrade	HTTP
Contract	Optional	Required	Required	None	Required
Shape	Request, response	Both, with streams	Request, response	Open both ways	Request, response
Browser can call it	Yes	Needs a proxy	Yes	Yes	Yes
Readable on the wire	Yes	No	Yes	Depends	Yes
Caching	Good	Poor	Poor	Not applicable	Poor

Public API, many unknown clients → REST
Service to service, low latency → gRPC
Clients needing different fields → GraphQL
Server sends without being asked → WebSockets
Enterprise system, strict rules → SOAP

8. Caching

Caching stores frequently used data temporarily so it can be retrieved faster.

A request served from a cache costs almost nothing.

A response can be held at several points on the way:

Browser → CDN → API gateway → Application → Database
↑ ↑ ↑ ↑
private shared shared in memory

HTTP does most of this for us, through headers on the response.

Header	What it does
Cache-Control	How long the response stays fresh, and who may keep it
ETag	A version tag for this exact response
If-None-Match	The client sending back the ETag it holds
Last-Modified	When the resource last changed
Vary	Which request headers change the response

The directives inside Cache-Control:

Directive	Meaning
max-age=60	Fresh for sixty seconds
no-cache	Store it, but check before using it
no-store	Never store it anywhere
private	Only the user's browser may keep it
public	Shared caches may keep it too
s-maxage=300	A separate age for shared caches

An ETag turns a repeat request into a cheap one:

```
HTTP/1.1 200 OK
Cache-Control: private, max-age=60
ETag: "a1b2c3"

{ "id": 1, "name": "Ravi", "salary": 75000.00 }
```

Next time the client sends the tag back:

```
GET /employees/1 HTTP/1.1
If-None-Match: "a1b2c3"
```

```
HTTP/1.1 304 Not Modified
ETag: "a1b2c3"
```

There is no body. The server still checked, and the network carried a few bytes instead of the whole resource.

Inside the application the cache is usually a library or Redis:

```
@Cacheable(value = "employees", key = "#id")
public Employee findById(long id) {
    return repository.findById(id).orElseThrow();
}

@CacheEvict(value = "employees", key = "#employee.id")
public void update(Employee employee) {
    repository.save(employee);
}
```

Four ways to arrange it:

Strategy	How it works
Cache aside	The app checks the cache, reads the database on a miss, then fills the cache
Read through	The cache reads the database itself on a miss
Write through	A write goes to the cache and the database together
Write behind	A write goes to the cache, and the database is updated later

Cache aside is the common one, and the others trade more coupling for fewer stale reads.

Invalidation is the hard part. A response is cached because it is expensive to build, and the same reason makes it expensive to notice it is wrong. Cache what is read far more often than it changes, and set the shortest age the product can accept.

Two failures to know by name. A **stale read** serves data that has changed, which is tolerable for a product description and not for a price or a permission. A **cache stampede** happens when a popular entry expires and every waiting request hits the database at once, which can be avoided by letting one request rebuild it while the others wait, or by spreading expiry times.

Cache → trade freshness for speed, on purpose and with a limit

9. API versioning

API versioning maintains different versions of an API so changes don't break existing clients.

Once other teams call an API, any change we make can break them.

Not every change breaks a caller:

Change	Breaking
Adding an optional field to a response	No
Adding a new endpoint	No
Adding an optional request parameter	No
Removing a field	Yes
Renaming a field	Yes
Changing a field's type	Yes
Making an optional field required	Yes
Changing what a value means	Yes, and the worst kind: nothing fails, the answers are just wrong

Four places to put the version:

Where	Example	Trade
URL path	<code>/v1/employees</code>	Obvious and easy to route. The most common
Query parameter	<code>/employees?version=1</code>	Simple, but it mixes with real parameters
Custom header	<code>X-API-Version: 1</code>	Clean URLs, harder to try by hand
Accept header	<code>Accept: application/vnd.example.v1+json</code>	What HTTP intends, awkward in practice

`GET /v1/employees/1 HTTP/1.1`

gRPC and GraphQL avoid versions in different ways. In a `.proto` file each field has a number, so adding a field is always safe and the rule is simply never to reuse a number. GraphQL usually has no version at all: new fields are added, and old ones are marked deprecated while both work.

```
type Employee {  
  name:      String!  
  salary:    Float  @deprecated(reason: "Use compensation")  
  compensation: Money  
}
```

Removing a version is the part that gets skipped. Announce it, say when it ends, and let the response carry the notice so a caller finds out without reading an email:

```
HTTP/1.1 200 OK  
Deprecation: true  
Sunset: Wed, 31 Dec 2026 23:59:59 GMT  
Link: </v2/employees>; rel="successor-version"
```

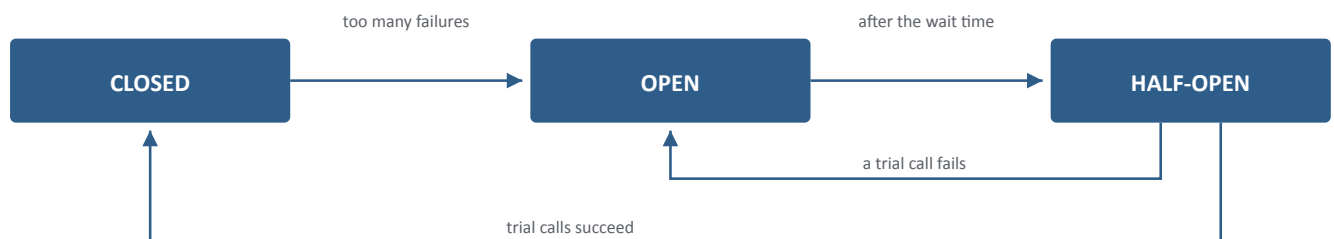
Add freely, remove slowly, never change a meaning in place

10. Circuit breaker

A circuit breaker stops sending requests to a failing service temporarily to prevent further failures.

When a service we depend on becomes slow, our own threads sit waiting for it. Once they are all waiting, we are down too, and so is everything calling us.

It has three states.



Open is the useful state: calls fail at once instead of waiting for a service that is already struggling.

State	What happens to a call
Closed	It goes through. Failures are counted
Open	It fails immediately. The dependency is not called at all
Half-open	A few trial calls are allowed, and they decide the next state

With Resilience4j in Spring:

```
@CircuitBreaker(name = "paymentService", fallbackMethod = "paymentUnavailable")
public PaymentResult charge(Order order) {
    return paymentClient.charge(order);
}

private PaymentResult paymentUnavailable(Order order, Exception e) {
    return PaymentResult.queued(order.id());
}
```

The fallback is what makes it useful. Failing fast with no answer is still a failure; queuing the payment and telling the customer it is pending is a service that still works.

```
resilience4j.circuitbreaker:
  instances:
    paymentService:
      failureRateThreshold: 50
      slidingWindowSize: 20
      waitDurationInOpenState: 30s
      permittedNumberOfCallsInHalfOpenState: 3
```

A circuit breaker is one of a family, and they are usually used together:

Pattern	What it does
Timeout	Gives up on a call after a fixed time
Retry	Tries again, waiting longer each time
Circuit breaker	Stops calling a dependency that keeps failing
Bulkhead	Caps concurrent calls so one dependency cannot take every thread
Fallback	Returns a reduced answer instead of an error
Rate limiter	Caps how many calls we make or accept

Retry without a circuit breaker makes an overload worse. Every client retrying a struggling service multiplies the load on it. The order that works is a timeout first, then a limited retry, with a circuit breaker above both.

Circuit breaker → fail fast, recover on purpose, do not spread the outage

Everything together

The style decides how two systems talk. The rest decide whether the API survives being used. Everything about proving who is calling is in the companion document, API Security.

Piece	What it answers
The style	How the two sides talk, and in what shape
Caching	Can we skip this work entirely
Versioning	Can we change this without breaking a caller
Circuit breaker	What do we do when a dependency stops answering

For example, imagine a food delivery application. The mobile app talks to the backend over REST, because it is public and caches well. The order service and the payment service talk over gRPC, because both are ours and the calls are frequent. The internal dashboard uses GraphQL, because each screen wants a different set of fields. Live driver location arrives over a WebSocket. The partner bank, which was integrated years ago, is reached over SOAP.

The restaurant list is cached at the CDN for a minute, because it is read constantly and changes slowly. The API is on `/v2`, and `/v1` still answers with a `Sunset` header while two partners finish migrating. The call to the payment provider sits behind a circuit breaker, so an outage there queues orders instead of holding every thread in the order service.

These are not exclusive. One system usually runs several.

The most useful thing at this point is not to memorize the tables. Take one endpoint we already call and write down what would change if it were gRPC instead: who generates the client, what the payload looks like, and what happens when a field is added.